Closing the Gap: Automated Discovery of Secure Dockerfile Reference Standards via Semantic Clustering in Enterprise Inner Source

Jessica Hösl
Technical University of Munich
Munich, Germany
jessica.hoesl@tum.de

Benedikt Hofmann
Siemens AG
Munich, Germany
hofmann.benedikt@siemens.com

Patrick Stöckle✉
Siemens AG
Munich, Germany
patrick.stoeckle@siemens.com

Abstract

Containerization dominates enterprise software delivery, yet Dockerfiles that assemble container images frequently harbor security misconfigurations and structural technical debt. This problem is poorly understood in corporate inner-source environments, where proprietary context and isolated governance prevent direct application of open-source findings.

We present an automated, six-stage pipeline that: (1) crawls an enterprise GitLab instance, (2) enriches each Dockerfile with static security and quality metrics (Hadolint, ShellCheck, Trivy) and lifecycle data, (3) groups functionally identical workloads using LLM-generated semantic descriptions and HDBSCAN, and (4) quantifies the optimization gap against cluster-internal reference implementations.

Applied to 11,470 Dockerfiles from over 6,200 repositories at a single large industrial company, we find a systemic deficit: 99% of files contain at least one security misconfiguration, 80.8% violate Dockerfile best practices, and the median artifact has not been revised for 838 days. Despite this, high-quality reference implementations already exist within 83% of functional clusters. Adopting these internal standards would increase the average security posture score by 60.4% without developing any new templates. These findings, grounded in one organization's inner-source ecosystem, provide a data-driven foundation for future automated, context-aware recommender systems targeting enterprise supply-chain security; whether the observed technical-debt distribution and optimization gap generalize to other enterprises remains an open question for future multi-organization study.

Keywords

Dockerfile security, technical debt, inner-source, mining software repositories, LLM, semantic clustering, container security, DevSecOps

1 Introduction

Containerization has become the cornerstone of modern software delivery, with industry surveys reporting over 90% adoption in production environments [6]. Dockerfiles—the configuration files that assemble container images—serve as executable blueprints for production workloads, CI/CD pipelines, and development environments alike. Writing secure, maintainable Dockerfiles is non-trivial: the format conflates system administration, shell scripting, and dependency management into a single artifact, demanding expertise that application developers often lack [16]. The result is a documented accumulation of structural *smells*, security misconfigurations, and lifecycle neglect in the broader ecosystem [5, 9, 19].

The enterprise inner-source gap. Virtually all large-scale empirical container research targets public GitHub or Docker Hub data [5, 8, 12, 19]. Enterprise inner-source environments—where thousands of teams share code via an internal code hosting platform (e.g., GitLab, Bitbucket) under proprietary guidelines— present a fundamentally different context. Internal images often embed company-specific hardened base images, carry organizational naming conventions, and lack the community governance present in popular open-source projects. Consequently, the volume, distribution, and remediability of security debt within a large corporation's container ecosystem has not been systematically studied.

Problem. The absence of a methodology to group enterprise Dockerfiles by *functional intent* prevents meaningful cross-team benchmarking. Without such grouping, it is impossible to identify which high-scoring internal configurations could serve as reference standards for their lower-quality functional peers. AI-driven remediation systems [11, 17, 20] are the most promising vehicles for scaled optimization, relying on techniques like In-Context Learning (ICL) and Retrieval-Augmented Generation (RAG) to automate repairs. However, to successfully generate context-aware code, these models require high-quality, domain-specific reference demonstrations, which do not exist in public datasets and are currently buried in organizational silos.

Contributions. This paper presents an automated six-stage pipeline deployed in the context of an internal GitLab instance of a major industrial company.

(1) **Semantic clustering methodology.** Enterprise Dockerfiles grouped by workload type based on LLM-generated functional descriptions of their intent, decoupling intent from syntactic implementation.
(2) **Composite Security Posture Score.** Linter warnings, security scans, base image footprint, and lifecycle metrics are unified into a normalized $[0, 1]$ ranking that quantifies the security and maintainability.
(3) **First enterprise-scale empirical characterization.** 11,470 Dockerfiles across 251 functional clusters, with a quantified optimization gap of 60.4% achievable from existing internal data.

2 Background and Related Work

2.1 Dockerfile Quality and Security Debt

An extensive body of mining software repository research has characterized the open-source Dockerfile landscape. Cito et al. identified missing version pinning as the dominant smell in 70,000+ GitHub Dockerfiles [5]; Wu et al. confirmed widespread smell co-occurrence in 6,334 projects [19]; Eng and Hindle revisited these findings with 9.4M Dockerfiles across seven years, observing slow but measurable quality improvement [8].

On the security side, Haque and Babar found that 91.6% of open-source projects inherit vulnerabilities from base images [9], and Shi et al. recently extended this to 93.7% of influential Docker Hub images [18]. Opdebeeck et al. showed that 70% of child images use a parent more than five months out of date [13]. Rosa et al. established a taxonomy of 46 optimization strategies by mining 1,026 improvement commits [14], while Ksontini et al. catalogued Docker-specific technical debt types from 68 open-source projects [10].

However, none of these studies target corporate inner-source environments, where different base image profiles, organizational standards and requirements (e.g., license compliance), and isolated development silos may lead to a distinct distribution of smells, misconfigurations, and maintenance patterns. Importantly, corporate environments also alter how the findings of standard tools such as Hadolint and Trivy should be interpreted. Company-hardened base images may pre-address certain Trivy security checks at the registry level—for example, a proprietary base image may already enforce a non-root user—while introducing internal security policies that tool defaults do not recognize. Internal package mirrors can render Hadolint version-pinning warnings (e.g., `DL3008`) misleading when internal mirrors guarantee a fixed dependency resolution. Organizational naming conventions cause Trivy and Hadolint to misclassify base images that use private registry prefixes, inflating false-positive counts. These factors collectively mean that open-source findings cannot be directly extrapolated to the corporate setting: an empirical study targeting a corporate inner-source instance is necessary to establish a valid baseline.

2.2 Automated Repair and LLM-Based Approaches

Rule-based tools such as DockerCleaner [3] and Parfum [7] automate correction of specific smells but cannot resolve complex, context-dependent misconfigurations. LLM-based approaches address this gap: Ksontini et al. use In-Context Learning (ICL) with GPT-4o, achieving 32% median image-size reduction [11], while LLMSecConfig [20] repairs Kubernetes misconfigurations with 94% success. Rosa et al. trained a T5 model on 670,000 Dockerfiles for full generation from requirements, but found it struggles with complex enterprise configurations [15].

A critical limiting factor for ICL systems is retrieval corpus quality. Ksontini et al. use BM-25 keyword matching [11], which cannot recognize functionally equivalent but syntactically diverse configurations. Zhang et al. showed that AST-based embeddings improve base-image recommendation but remain tied to structural syntax [21].

Enterprise codebases are substantially more heterogeneous than open-source repositories: different teams independently adopt different base images, package managers, and build patterns for the same workload type, making syntactic similarity a poor proxy for functional equivalence. High-Level Specification (HLS) grouping by base image and installed packages partially addresses this but remains anchored to implementation artifacts: it conflates functionally distinct workloads that share a common base image (e.g., a Python web API and a Python data-science notebook both built on `python:3.11`), while splitting functionally identical containers that happen to use different distributions (e.g., Alpine vs. Debian for the same REST service). Applying Hadolint and Trivy directly to the full corpus yields ecosystem-wide statistics (RQ1) but cannot identify *which* specific Dockerfile should serve as a reference standard for *which* other Dockerfile—the precise mapping that automated remediation and knowledge-transfer tools require. Semantic clustering provides this mapping by grouping on functional intent rather than implementation.

2.3 Relationship to Commercial Container Security Tooling

Commercial platforms such as Snyk Container, Aqua Security (the vendor behind Trivy, which we use directly in Stage 3), and JFrog Xray already provide enterprise-scale vulnerability scanning and remediation suggestions for container images. These tools excel at per-image, per-layer vulnerability detection and typically propose a single upstream fix (e.g., bump a base-image tag or a package version) grounded in CVE databases and vendor advisories. They operate, however, at the level of an individual image or dependency graph: they do not group functionally equivalent Dockerfiles across an organization's codebase, and they have no notion of an internally vetted *golden reference* that already satisfies a company's own hardening conventions, licensing constraints, and base-image standards. We view our contribution as complementary to, rather than competing with, this class of tools: their scan output (Trivy, in our pipeline) is a direct input to the Security Posture Score, while semantic clustering and golden-reference discovery add the missing organizational layer—identifying *which* already-compliant internal configuration should be propagated to *which* functionally equivalent peer, a mapping that commercial per-image scanners do not produce.

2.4 Research Gap

Three gaps remain: (1) *contextual*—no large-scale study targets corporate inner-source; (2) *methodological*—as discussed above, recommendation frameworks lack semantic awareness for functionally heterogeneous enterprise configurations, requiring grouping by intent rather than syntax to map specific remediation targets; (3) *foundational*—no approach systematically extracts high-quality internal reference implementations to seed automated remediation. Our work addresses all three.

3 Industrial Context and Problem

The study was conducted at a global industrial technology company with a self-hosted GitLab instance. Development is organized

across autonomous departments that can share code via inner-source[1, 2]. We only analyzed inner source repositories—those accessible to all employees—to capture the shared ecosystem where cross-team knowledge transfer is possible, which included approximately 44,000 repositories. Teams create their Dockerfiles or fork and extend base configurations but rarely re-synchronize with functional peers. This model creates *silos of quality*, i.e., a highly optimized, company-hardened .NET container built by a security-focused team coexists with dozens of misconfigured functional equivalents written by teams lacking that expertise, although the others could access and adopt the high-quality reference if they knew it existed.

Three concrete operational costs motivate automated remediation:

Attack surface. Each container running as root or inheriting an unpatched base image is a potential lateral-movement entry point.

Maintenance burden. With dormant or abandoned Dockerfiles, engineering teams face growing hidden liability as upstream packages accumulate CVEs while configurations remain static.

Duplication of effort. Functional clusters contain dozens to hundreds of independent reimplementations of the same workload type. Systematic standardization can eliminate this redundancy, reducing the number of unique artifact configurations requiring audit and maintenance.

Prior to this work, no tooling existed to quantify this debt, map its distribution, or extract remediation targets from existing internal repositories.

Research Questions

This study is structured around three operationally motivated research questions:

RQ1 — Ecosystem State.
What is the current security posture of Dockerfiles in a large corporate inner-source environment, as characterized by static analysis metrics and lifecycle data? We establish the first empirical baseline for a corporate container ecosystem, quantifying smell prevalence, misconfiguration rates, and maintenance neglect.

RQ2 — Semantic Clustering Quality.
Can LLM-generated functional descriptions, combined with density-based clustering, effectively group enterprise Dockerfiles by workload type? We evaluate whether semantic grouping produces tighter functional cohesion than syntactic baselines—a prerequisite for valid intra-cluster benchmarking.

RQ3 — Addressable Remediation Potential.
How much of the measured security debt can be resolved through internal knowledge transfer, i.e., by promoting configurations that already exist within the same functional cluster? We quantify the per-file and ecosystem-wide optimization gap, and identify the high-priority clusters where automated remediation can have the largest measurable impact.

RQ1 is addressed in Section 5.3, RQ2 in Section 5.2, and RQ3 in Section 5.4.

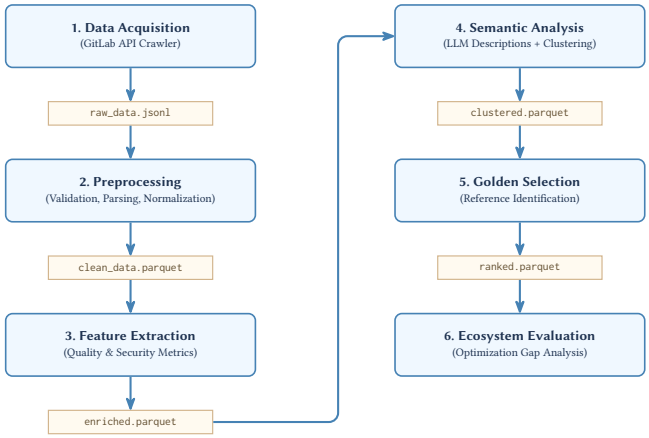

Figure 1: Six-stage pipeline with intermediate data artifacts. Each stage persists results as Parquet snapshots for reproducibility.

4 Approach

The pipeline consists of six sequential stages implemented in Python, with specialized tool invocations (Go-based Moby parser, Hadolint, ShellCheck, Trivy, crane) orchestrated centrally. Intermediate results are persisted as Apache Parquet snapshots for independent validation and reproducibility. Figure 1 illustrates the complete pipeline.

4.1 Stage 1: Data Acquisition

We collect the Dockerfiles using a custom crawler that interacts with the GitLab v4 API, targeting the default branch of each project. Named Dockerfile variants (e.g., `Dockerfile.base`) are included; template files (Jinja2 placeholders), documentation, and Windows-image Dockerfiles are excluded. The crawler collects: project metadata, CI/CD pipeline configuration, and container registry information.

Furthermore, Git commit history is extracted via partial cloning to capture file-specific revision timelines without downloading blob content to optimize bandwidth and storage. The resulting raw dataset comprised **12,002** unique Dockerfiles from 6,247 repositories.

4.2 Stage 2: Preprocessing and Normalization

Raw records undergo integrity checks (empty, malformed, invalid-syntax entries removed), yielding **11,470** cleaned Dockerfiles. Each file is parsed into an AST via a custom Go executable wrapping the official `moby/buildkit`[1] parser, ensuring syntax parity with the Docker build engine for heredoc and multi-stage syntax.

Build-time `ARG`/`ENV` variables are resolved from associated CI/CD configurations, reducing unresolved base-image variables by 20%. Normalized AST strings—comments stripped, multi-line `RUN` blocks concatenated—are stored for downstream analysis.

[1] github.com/moby/buildkit

4.3 Stage 3: Feature Extraction and Scoring

We compute six normalized metrics spanning three dimensions:

Static Code Quality. Hadolint[2] identifies Dockerfile-specific smells (DL-series rules). ShellCheck[3] analyzes concatenated `RUN` blocks, detecting cross-instruction shell errors that Hadolint misses in isolation. Both are normalized as *defect density* (violations per instruction), clipped at population 95th-percentile saturation points ($D^{\text{HL}}_{\max} = 1.11$, $D^{\text{SC}}_{\max} = 0.25$) to prevent extreme outliers from compressing the score of the addressable majority:

$$N_{\text{HL}}(d) = 1 - \min\left(\frac{v_{\text{HL}}(d)/n(d)}{D^{\text{HL}}_{\max}},\ 1\right)$$

where $v_{\text{HL}}(d)$ is the Hadolint violation count and $n(d)$ the instruction count of Dockerfile d. The ShellCheck component N_{SC} is computed analogously. The main reason for using density rather than raw counts is to avoid penalizing longer files that may naturally have more violations but are not necessarily of lower quality.

Security Configuration. Trivy[4] in configuration-scan mode identifies misconfigurations (e.g., `DS002`: running as root, `DS004`: exposing privileged ports) without building images. Raw counts $v_{\text{TV}}(d)$ are $\ln(x+1)$-transformed and saturated at 10 misconfigurations:

$$N_{\text{TV}}(d) = 1 - \min\left(\frac{\ln(v_{\text{TV}}(d)+1)}{\ln(11)},\ 1\right)$$

Running Hadolint and Trivy in parallel is intentional: the former flags stylistic violations (*explicit* `USER root`), the latter flags security state (*implicit* root inherited from a base image), capturing both noisy and silent risks.

Supply Chain and Lifecycle. Base image compressed size $s(d)$ (queried via crane[5]) is log-normalized over a $[s_{\min} = 10\,\text{MB}, s_{\max} = 2\,\text{GB}]$ range:

$$N_{\text{size}}(d) = 1 - \min\left(\frac{\ln(\max(s(d), s_{\min})) - \ln(s_{\min})}{\ln(s_{\max}) - \ln(s_{\min})}, 1\right)$$

Maintenance metrics are calibrated to literature-derived normative thresholds: *update frequency* (average revisions per year, $F_{\max} = 12$ rev/year [5]) and *recency* (days since last commit, $R_{\max} = 730$ days [9]). A higher revision frequency and lower recency (more recent) each contribute positively to the score.

The **Security Posture Score** aggregates these six metrics:

$$S = \sum_{i=1}^{6} w_i \cdot N_i(m_i), \qquad S \in [0, 1]$$

with weights listed in Table 1: Trivy and Hadolint each receive 0.25 (security risk and code quality), update frequency 0.20, base image size 0.15, recency 0.10, and ShellCheck 0.05. This configuration prioritizes immediately exploitable risks over stylistic issues.

To prevent the systematic penalization of technology stacks with inherently larger footprints, such as machine learning or robotics workloads, we compute a *Relative Security Posture Score* S_{rel} for intra-cluster comparison. This metric normalizes base image size using each cluster's specific 5th–95th percentile bounds, ensuring the resulting optimization gap reflects remediable configuration debt rather than architectural requirements.

[2] github.com/hadolint/hadolint
[3] shellcheck.net
[4] github.com/aquasecurity/trivy
[5] github.com/google/go-containerregistry

Table 1: Security Posture Score: weights and normalization limits.

Metric	w_i	Limit	Rationale
Trivy (security)	0.25	>10 misconfigs	Immediate exploitability risk
Hadolint (quality)	0.25	density >1.11	Prerequisite for secure builds
Update frequency	0.20	>12/year	Ability to respond to threats
Base image size	0.15	>2 GB (log)	Attack surface / bloat
Recency	0.10	>730 days	Staleness / vulnerability lag
ShellCheck	0.05	density >0.25	Shell-specific logic errors

4.4 Stage 4: Semantic Clustering

Why semantic descriptions? Grouping Dockerfiles by syntactic features over-indexes on implementation details. For example, two Python REST APIs built on Alpine/pip and Debian/conda would be split into separate clusters, while syntactically similar but functionally different configurations are merged. Inspired by limitations identified in prior retrieval work [11, 21], we embed natural-language *functional descriptions* instead.

LLM description generation. Each normalized Dockerfile AST is submitted to Qwen2.5-30B-Instruct with the following constrained prompt template:

> *Generate a Semantic Capability Summary for this Dockerfile.*
> *Constraints:*
> *1. Describe the Application Workload (e.g., 'Python Data Science', 'Node.js API') and Key Frameworks.*
> *2. Describe the Functional Intent (what the container does).*
> *3. NEGATIVE CONSTRAINT: Do NOT mention the Base Image OS (e.g. Ubuntu, Alpine, Debian).*
> *4. NEGATIVE CONSTRAINT: Do NOT mention specific version numbers.*
> *5. NEGATIVE CONSTRAINT: Do NOT mention build-time cleanup or security flags.*
> *6. Output exactly 2-3 concise sentences.*

The LLM is internally hosted to avoid data leakage and ensure compliance with the company's data governance policies. Temperature $T = 0.1$ minimizes output variance across repeated runs. Files exceeding 9,000 characters are truncated in the middle, preserving the `FROM` header and `CMD`/`ENTRYPOINT` instructions to retain workload-identification signal. The exclusion of OS distribution and version numbers is deliberate: including them would allow the embedding model to cluster by implementation artefacts rather than by functional intent, defeating the semantic grouping objective.

Embedding and clustering. Descriptions are embedded with `all-mpnet-base-v2` (Sentence-Transformers, $\mathbb{R}^{768}$). Dimensionality is reduced to 50 via UMAP (cosine metric, $k = 15$, min_dist $= 0$). HDBSCAN (minimum cluster size 15, minimum samples 3) identifies dense functional groups; sparse points are labelled as noise [4].

We selected HDBSCAN over partitioning methods (e.g., k-means) and agglomerative clustering for three reasons specific to an enterprise Dockerfile corpus of unknown functional granularity. First, the number of distinct workload types is not known a priori and

cannot be reliably estimated in advance; HDBSCAN does not require the target cluster count k to be fixed, unlike k-means. Second, functional cluster sizes are highly imbalanced (from single-digit niche workloads to the 494-file ASP.NET cluster in Table 2); density-based clustering accommodates this variable density better than centroid-based methods, which implicitly assume roughly spherical, similarly sized clusters. Third, LLM-generated descriptions for atypical or highly customized Dockerfiles are occasionally ambiguous or idiosyncratic; HDBSCAN's explicit noise label lets such points remain unclustered rather than forcing them into an ill-fitting group, which would otherwise silently corrupt intra-cluster comparisons. We did not empirically ablate this choice against alternative clustering algorithms on the full corpus; Section 7 discusses this and related methodological robustness questions as an explicit threat to validity.

4.5 Stages 5–6: Golden Selection and Optimization Gap

Within each cluster C_k, the *golden reference* g_k is the Dockerfile maximizing S_{rel}. The *Optimization Gap* for any file $d_i \in C_k$ is:

$$\Delta_{\text{opt}}(d_i) = S_{\text{rel}}(g_k) - S_{\text{rel}}(d_i), \quad g_k = \arg\max_{j \in C_k} S_{\text{rel}}(d_j)$$

A *conservative P90 target* replaces g_k with the 90th-percentile intra-cluster score, accounting for configurations that score highly only through extreme architectural constraints (e.g., `FROM scratch` static binaries) that cannot be generalized across the full cluster.

5 Evaluation

5.1 Dataset Overview

The final dataset contains **11,470** Dockerfiles from **6,247** repositories. The top 20 base images cover 67.3% of all files; `python`, `node`, `alpine`, and `ubuntu` are the four most prevalent. Base image sizes span from virtually empty (`scratch`) to 10.75 GB, with a median of 47 MB and a 95th percentile of 471 MB.

The industrial base-image profile differs markedly from public-repository studies [12]: `aspnet` ranks first by cluster size (ahead of `python` and `node`), reflecting the dominance of enterprise web services, and two robotics/embedded clusters have no meaningful counterparts in open-source datasets. Table 2 presents the ten most populous functional clusters, spanning six distinct technology stacks that collectively represent 17.5% of the dataset.

Beyond lifecycle concerns, the ecosystem's architectural footprint reveals significant differences in base image usage, which directly affect operational efficiency and attack surface. The top 20 base images account for 67.3% of all Dockerfiles, with `python`, `node`, `alpine`, and `ubuntu` being the most prevalent (Figure 2). Base image sizes exhibit a right-skewed distribution: the median is 47 MB (indicating a preference for optimized images), while the 95th percentile reaches 471 MB and the maximum extends to 10.75 GB (a machine learning `acpt-pytorch-cuda` image). This underscores the need for context-specific optimization: 50 MB is substantial for `alpine`-based microservices but negligible for GPU workloads.

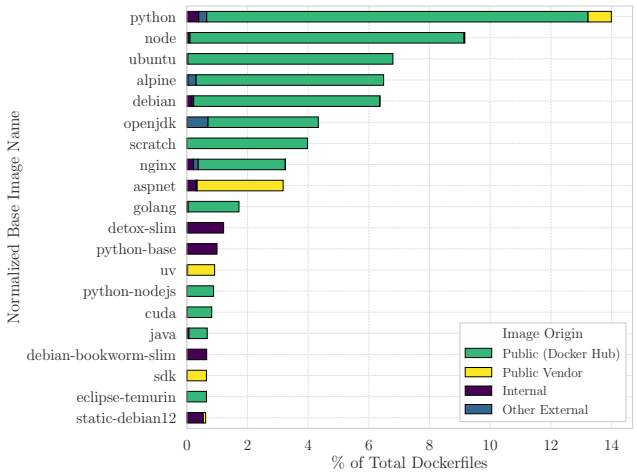

Figure 2: Distribution of base image sizes across the dataset.

5.2 Clustering Validation

Semantic clustering produced **251 clusters** covering 83% of the dataset (17% noise, i.e., points not assigned to any cluster). Table 3 reports cluster-size weighted internal cohesion metrics.

Although the semantic approach yields higher base-image entropy than syntactic baselines (1.65 vs. 0.79–1.20), this is by design: the model groups heterogeneous implementations of the same workload. The global dataset entropy is 4.32; semantic clustering reduces it by **62%**, confirming meaningful functional cohesion. The raw-content baseline produces 21% noise, indicating it over-fits to syntactic variation.

We emphasize that Table 3 isolates the effect of the *input representation* (semantic descriptions vs. raw content vs. HLS) while holding the clustering algorithm (HDBSCAN) fixed; it is not a comparison across clustering algorithms. Whether an alternative algorithm (e.g., k-means or agglomerative clustering) applied to the same semantic embeddings would yield comparable or superior cohesion remains untested on this corpus and is discussed further in Section 7.

The force-directed graph visualization in Figure 3 provides both topological and evaluative perspective on the clustered ecosystem. Nodes represent individual Dockerfiles, and edges denote cosine embedding similarities exceeding 0.6—a threshold chosen to reduce visualization noise while preserving the distinct topological structure of functional groups.

The resulting topology reveals tightly connected color-coded islands (dense functional clusters), validating the algorithm's ability to isolate semantically coherent workloads. For example, the large violet cluster on the left (494 Dockerfiles, .NET applications) correctly groups canonical `aspnet` images and company-hardened enterprise derivatives by their shared runtime purpose, despite different registry origins and security scores (0.07–0.87).

Similarly, Node.js workloads are close together spatially but subdivided into semantically distinct groups: frontend frameworks (React, Next.js) are separated from backend API gateways and build environments. The clear, visually distinct grouping confirms that

Table 2: Top 10 functional clusters by file count. Dom. = dominant base image and its share within the cluster. Together these ten clusters cover 17.5% of all Dockerfiles across six distinct technology stacks.

ID	N	Dom. Base Image	Functional Domain
27	494	`aspnet` (61%)	ASP.NET Core web applications and REST API services
215	262	`python` (24%)	ML model development and inference (TensorFlow/PyTorch)
191	196	`ubuntu` (26%)	C/C++ compilation and build toolchains
169	172	`debian` (40%)	Embedded systems C/C++ cross-compilation (ARM)
147	147	`alpine` (16%)	Kubernetes operations and infrastructure automation (Helm)
62	132	`ros` (23%)	Real-time robotics and sensor processing (ROS 2)
151	101	`tomcat` (20%)	Identity, authentication, and secure token management
210	86	`alpine` (34%)	High-performance compiled Go microservices
76	84	`python` (17%)	Security scanning and automated vulnerability assessment
143	82	`nginx` (43%)	Frontend web applications and static asset delivery

Table 3: Clustering method comparison. Semantic (ours) is evaluated against syntactic baselines using Dockerfile Content (raw text) and HLS (High-Level Specification: base image + installed packages) as input representations.

Method	#Clusters	Noise	Img Entropy	Pkg Overlap
Semantic (ours)	251	0.17	1.65	0.41
Content (raw)	257	0.21	1.20	0.46
HLS (base+pkgs)	309	0.14	0.79	0.59

semantic descriptions decouple functional intent from syntactic implementation.

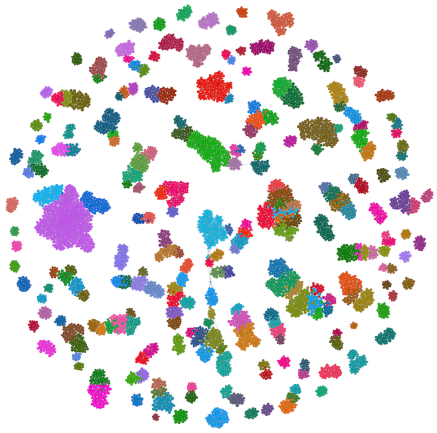

Figure 3: Force-Directed Graph of the Dockerfile Ecosystem, colored by Cluster ID. Nodes represent individual Dockerfiles; edges denote cosine embedding similarity > 0.6. Distinct topological clusters validate the semantic model's ability to isolate functionally coherent workloads.

Figure 4 shows the *same* force-directed topology annotated by relative score instead of cluster membership. Each node remains a Dockerfile; the topology is invariant. The color gradient from dark (low S_{rel}) to light (high S_{rel}) reveals the within-cluster quality variance: in most clusters, a small subset of light nodes coexists with a large majority of dark and medium nodes, visually confirming the bi-modal quality distribution that the Optimization Gap metric captures numerically.

Figure 4: Force-directed graph of the clustered Dockerfiles, colored by relative security posture score (dark = low, light = high). Within-cluster score spread is visible across virtually all functional groups, confirming that high-quality configurations rarely propagate to their lower-scoring cluster peers.

5.3 State of the Ecosystem

Lifecycle neglect. Figure 5 illustrates the recency distribution and Figure 6 the revision-count distribution. Both are strongly right-skewed. While 30% of files where updated within the last year, the median Dockerfile has not been revised in **838 days** (2.3 years); the oldest unmaintained file has not been updated for a decade. While a lack of recent commits can indicate a stable, feature-complete service rather than abandonment, this extreme latency creates severe security risks. In containerized environments, an unmaintained Dockerfile could pull outdated, unpatched base OS layers and dependencies, leading to software decay, even when the application code remains unchanged. Applying a maintenance taxonomy:

- **Dormant/Abandoned** (last commit >1 year): **70.2%**
- **Actively Maintained** ($\geq$ 2 rev/year, recent): **23.4%**

- Stable or New: 6.4%

Update frequency mirrors open-source findings [5]: half the ecosystem is updated once or less per year, yet the top 10% exceed 9.8 revisions annually. The bimodal character suggests that automated remediation would disproportionately benefit the inactive majority.

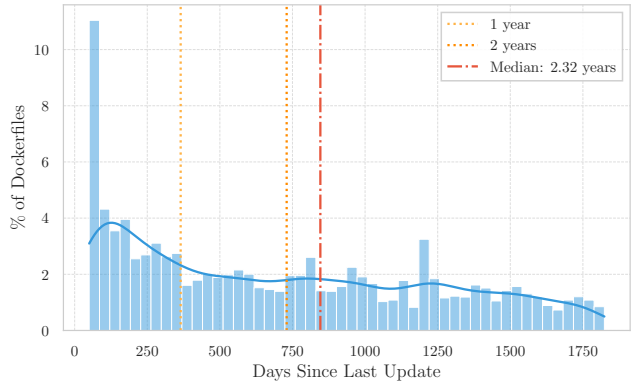

Figure 5: Distribution of artifact recency (days since last update). 70.2% of the ecosystem has not been updated in over a year.

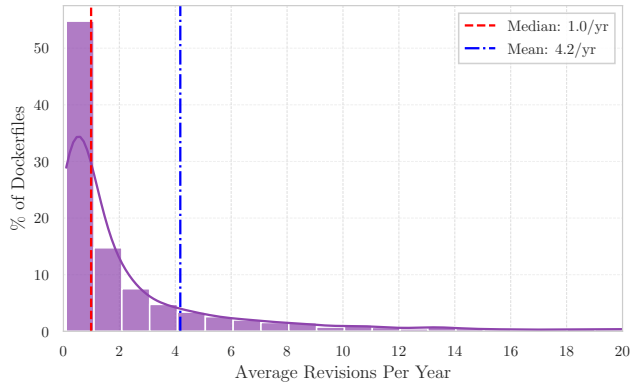

Figure 6: Distribution of Git revision counts per Dockerfile.

Smell prevalence. Hadolint identified **45,902 violations** across **80.8%** of all Dockerfiles (median: 2; maximum: 95). While these violations are widespread, their distribution is highly concentrated. The most common violations by prevalence are missing version pinning (`DL3008`, 29.4% of files), consecutive `RUN` instructions (`DL3059`, 27.4%), and the absence of `-no-install-recommends` (`DL3015`, 19.0%).

Figure 7 shows the top-10 violations by prevalence. The data identifies two primary categories of technical debt: Layer Optimization (e.g., `DL3059`, `DL3015`) and Package-manager Pinning (`DL3008`, `DL3013`, `DL3016`). Together, these categories affect over 50% of the total files in the dataset. While pinning rules are a critical component of supply-chain security, their prevalence (appearing in 5,269 files) is matched by optimization smells (5,322 files). This indicates that developers struggle equally with the security and the efficiency of their Dockerfiles.

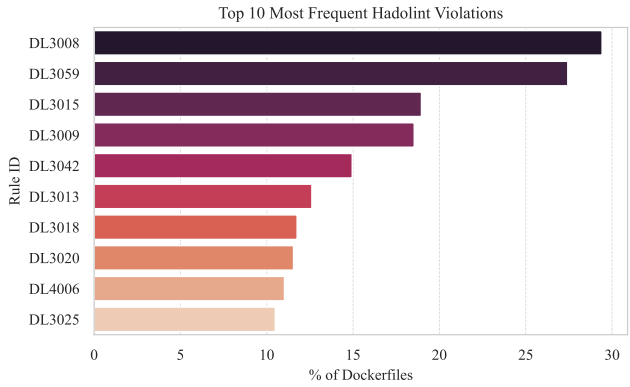

Figure 7: Top-10 Hadolint rule violations. Percentages reflect the prevalence of each rule across the total dataset ($N = 11,470$).

Security misconfigurations. Trivy flagged **99%** of Dockerfiles with at least one misconfiguration (Figure 8). The dominant pair—missing `HEALTHCHECK` (`DS026`) and running as root (`DS002`)—appears simultaneously in **81.7%** of files, producing a spike at exactly two misconfigurations (42.4% of the dataset). Running as root (`DS002`), present in 81.7% of files, is particularly concerning: if a container process is compromised, an attacker would have root privileges within the container, indicating that foundational security controls are systematically deferred to downstream orchestration layers rather than embedded at the source.

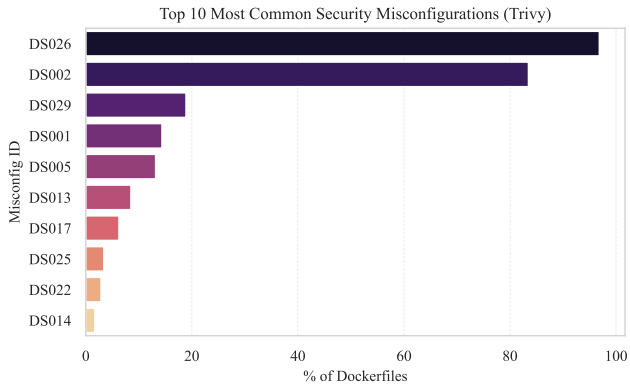

Figure 8: Top-10 Trivy misconfigurations. `DS026` (missing HEALTHCHECK) and `DS002` (root user) co-occur in 81.7% of files.

Composite score. The ecosystem achieves a mean S_{rel} of **0.48** ($SD = 0.16$, median 0.47)—centered near the midpoint of the $[0, 1]$ scale. The distribution is approximately normal with slight left skew: approximately 6.3% of files score above 0.75 ("acceptable security

Table 4: Optimization gap by clustering method.

Method	Total debt	Avg/file	Rel. improvement
Semantic (ours)	2784.5	0.29	60.4%
Content (raw)	2212.4	0.24	49.6%
HLS	2244.3	0.23	46.4%

posture") and only 1.2% exceed 0.85 ("strong posture"), while 11.1% score below 0.30. This global symmetry masks important cluster-level heterogeneity: the ML cluster (ID 215) has a mean of 0.40, while the Go microservices cluster (ID 210) averages 0.56. The weighted standard deviation within clusters is 0.13 on average, confirming that meaningful intra-cluster spread—rather than across-cluster divergence—is the primary driver of the optimization gap.

5.4 Security Debt and Optimization Gap

Core result. For the 83% of Dockerfiles successfully clustered, the mean optimization gap is $\overline{\Delta}_{\text{opt}} = 0.29$. The average clustered Dockerfile can increase its absolute score by **29 percentage points**—a **60.4% relative improvement**—simply by adopting the best practices already present in its functional cluster. The most severely under-performing 10% face a gap exceeding 0.53: alignment with the golden reference would more than double their security posture.

Table 4 compares optimization gaps across clustering methods. Syntactic methods report a substantially lower gap (49.6%/46.4%) because they trap insecure files with structurally similar but equally insecure neighbors, hiding the existence of superior functional equivalents. By grouping on intent rather than syntax, semantic clustering compares configurations that serve the same purpose, regardless of implementation differences.

Ecosystem-wide impact. If all clustered debt were resolved, the global mean score (all $N = 11,470$ files, including noise) would rise from 0.48 to **0.72**—a **50.6% global improvement**. The conservative P90 target yields 34.5% relative improvement for clustered files and a **28.75% global improvement**—still a substantial and operationally realistic gain.

Cluster-level variance. Figure 9 visualizes the score distribution and golden reference for the 15 largest clusters. The persistent gap between each cluster's top quartile and its golden reference (★) is visible across all workload types. Within Kubernetes infrastructure automation (C-147), security scores span the full range from near-zero to near-perfect, while the dominant base image (`alpine`) accounts for only 16% of the cluster; the remaining 84% perform similar operational tasks on different images. This confirms that measured debt is not a property of a few outlier workloads but is deeply embedded across all major categories.

High-priority remediation clusters. Analysis of the bottom-15 clusters by mean score reveals two failure patterns. *Lifecycle abandonment*: Python DevOps tooling (C-148) and JavaScript QA automation (C-103) have median recencies of 1,384 and 1,246 days respectively. While both exhibit extreme age, their technical debt profiles diverge: C-148 contains a median of 61 Hadolint violations per file, whereas C-103 reflects a lower but still elevated median of 9 violations. *Architectural sprawl*: Robotics workloads (C-62) exhibit an extreme lack of standardization, with base image footprints

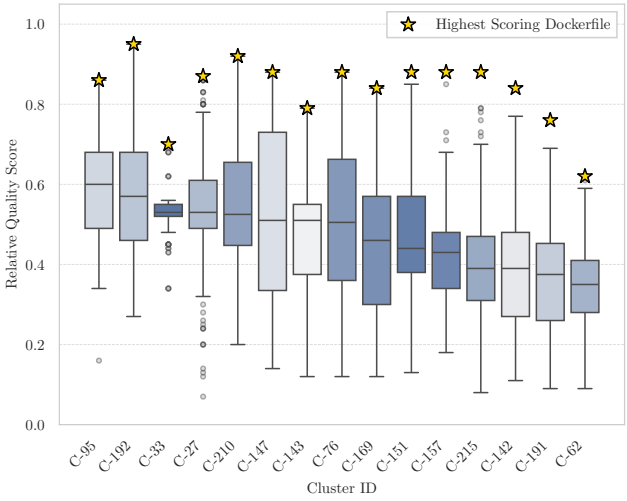

Figure 9: Security Posture Score distributions vs. maximum reference scores for the 15 largest clusters. Stars (★) mark golden references. The persistent gap confirms systemic, not isolated, debt.

spanning three orders of magnitude (26 MB to 8.2 GB), indicating the absence of a unified engineering standard. Flink/Kafka stream-processing (C-73) shows a slightly more contained variance (28 MB to 535 MB).

Table 5 presents the seven clusters with the highest cumulative optimization gap that also contain an internal reference standard with $S_{\max} \geq 0.80$. These represent the highest-priority targets for automated remediation: rich in debt, yet already possessing an good reference configuration. The ASP.NET cluster (ID 27) alone concentrates 156.95 cumulative gap points across 494 files; closing it costs zero new template development. The ML inference cluster (ID 215) is the second-largest opportunity, with a median score of only 0.39 despite the cluster's best configuration reaching $S_{\max} = 0.88$—a gap that reflects the widespread practice of prototyping ML workloads without hardening before deployment.

Clusters without viable references. The top 20 functional clusters have a median structural baseline (median $S_{\max}$) of 0.90. Conversely, the bottom 20 clusters currently lack any reference implementation approaching a standard, meaning no internal candidate is suitable for automated remediation. These clusters consist largely of near-identical clones of a single low-quality template, duplicated before a secure baseline was ever established. A targeted investment in a single hardened base image per orphaned cluster would increase the total addressable optimization potential by 7.7% (215 cumulative debt points across 422 files).

Table 6 identifies the five most important active clusters in this category—active in the sense that recent commit history confirms ongoing engineering investment. These are not abandoned workloads; they represent live deployments lacking a quality anchor. The Internal Tool cluster (ID 12) is particularly notable: with a median recency of 197 days, it is the most actively maintained orphaned cluster, yet no member has reached $S_{\max} = 0.75$. This

Table 5: High-yield remediation targets: clusters with the highest cumulative optimization gap that already contain a viable internal reference ($S_{max} \geq 0.80$). $\sum \Delta_{opt}$ is the total addressable security debt within the cluster.

ID	N	Functional Domain	S_{med}	S_{max}	$\sum \Delta_{opt}$
27	494	ASP.NET Core web applications and REST API services	0.53	0.87	156.95
215	262	ML model development and inference (TensorFlow/PyTorch)	0.39	0.88	125.94
169	172	Embedded systems C/C++ cross-compilation (ARM)	0.46	0.84	66.05
147	147	Kubernetes operations and infrastructure automation (Helm)	0.51	0.88	52.12
151	101	Identity, authentication, and policy enforcement (OAuth2)	0.44	0.88	42.89
157	81	C/C++ development environments and Python build automation	0.43	0.88	36.38
142	74	AWS infrastructure automation and cloud governance tooling	0.39	0.84	32.41

Table 6: Active clusters without a viable internal reference ($S_{max} < 0.75$). Despite recent development activity, no cluster member has reached sufficient quality to serve as a remediation template. Recency = median days since last commit.

ID	Functional Domain	N	S_{max}	Recency
45	Java Spring Boot (JAR)	39	0.56	245 d
133	NGINX Static Frontend	18	0.62	265 d
214	Node.js/TS Build & Test	22	0.66	355 d
12	.NET/Node.js Internal Tool	29	0.67	197 d
63	Java JVM (container-opt.)	28	0.67	245 d

signals a systematic organizational gap—one that a purpose-built, security-reviewed image could resolve for all current and future users.

6 Lessons Learned and Industrial Implications

L1: Inner-source has structurally higher but unevenly distributed maintenance activity. The top 10% of inner-source Dockerfiles receive nearly 10 revisions per year due to close coupling to production workloads. Yet 70% remain dormant. The enterprise setting does not immunize organizations from the systemic infrastructure neglect documented in public repositories [5, 8].

L2: The problem is propagation, not expertise. The 60.4% optimization gap does not reflect a lack of internal knowledge. High-quality, hardened configurations already exist within 83% of functional clusters—built by security-aware teams. The problem is that other teams writing functionally equivalent containers never discover these standards. This is exactly the retrieval problem that a semantics-aware recommender can solve.

L3: Semantic clustering yields a 14-percentage-point improvement over the best syntactic baseline, and the LLM cost is a one-time batch investment. As seen in Table 4, HLS-based clustering—the strongest syntactic baseline—reports a 46.4% relative optimization gap; raw-content clustering reports 49.6%; our semantic approach reports 60.4%. The 14-percentage-point improvement over HLS arises because semantic clustering correctly bridges functionally equivalent containers that differ only in their implementation choices (OS distribution, package manager), exposing high-quality references that syntactic methods never associate with their lower-quality functional peers. The cost of this improvement is the one-time batch inference of LLM-generated functional descriptions for the 11,470-file corpus using an internally hosted model (Qwen2.5-30B-Instruct). Because the model is self-hosted, there is no per-token billing; the computation is a bounded, single-pass job that does not recur unless the corpus is refreshed. In contrast, the 14-percentage-point shortfall of HLS clustering is a *permanent structural deficit*: no amount of additional syntactic refinement can recover functionally equivalent but syntactically dissimilar configurations. For a 60% improvement over an 11,000-file corpus, we conclude that the one-time LLM inference cost is justified; practitioners in resource-constrained environments may nonetheless opt for HLS clustering as a lower-cost approximation that still captures 46% of the addressable debt. Practitioners deploying ICL-based repair systems with syntactic retrieval [11] leave this meaningful improvement unrealized by failing to bridge the gap between distinct-but-equivalent configurations.

L4: 83% of the debt is addressable from existing data; semantic clustering identifies 30% more addressable debt per file than the syntactic baseline. Large organizations need not commission new secure templates to begin remediation. To quantify how much of this result depends on semantic clustering specifically: under HLS-based clustering, 86% of files are assigned to a cluster (vs. 83% for semantic) but the mean addressable gap per file is only 0.23 (vs. 0.29 for semantic)—a 20.7% reduction in identified optimization potential per artifact. In absolute terms, HLS clustering surfaces 2,244 cumulative debt points against semantic clustering's 2,785—a difference of 541 debt points that would remain hidden without functional grouping. The additional coverage comes from configurations that HLS incorrectly separates or merges, causing their actual best functional-peer reference to be invisible to the analysis. Existing internal best practices are thus sufficient to address the overwhelming majority of identified debt under either method; the semantic approach ensures that more of that debt is correctly attributed and that a richer set of valid reference implementations is surfaced. Only the minority of orphaned clusters require targeted

engineering investment—and this analysis has identified precisely those clusters.

Enabling downstream automation. The enriched, clustered dataset is a direct input to LLM-based repair systems [11, 20]: golden references serve as in-context demonstrations for enterprise-native repair suggestions; semantic cluster labels enable workload-aware retrieval, directly addressing the BM-25 limitation; and the Optimization Gap provides a prioritized list of files warranting automated intervention.

7 Threats to Validity

Construct. The Security Posture Score is a proxy for configuration hygiene, not a runtime CVE count. Equal weighting of all warnings regardless of severity may over-penalize low-severity issues. Weights and thresholds reflect expert judgment and are subject to sensitivity analysis. They were calibrated to align with industry best practices and internal security standards, but alternative configurations could yield different absolute scores.

Internal. Variable resolution reduced unresolvable base-image definitions to 5.1%; these files were retained to avoid bias against complex, parametrized Dockerfiles. The Git `--follow` flag was omitted, potentially affecting recency for renamed files. Hadolint's stricter parser may have discarded a small number of buildable but non-standard Dockerfiles.

External. The dataset originates from a single corporate environment. Technical debt distribution, workload mix, and governance posture may differ across organizations. The study is a static snapshot; longitudinal analysis of debt accumulation velocity remains future work.

Clustering. No labeled ground truth is available for unsupervised clustering on proprietary data. We validated relative cohesion via base-image entropy and package Jaccard similarity against syntactic baselines. UMAP and HDBSCAN hyperparameters were tuned heuristically; different settings alter cluster granularity and the measured Optimization Gap. Our clustering-algorithm ablation is limited to input representation (Table 3): we did not compare HDBSCAN against alternative clustering algorithms (e.g., k-means, agglomerative, or spectral clustering) applied to the same semantic embeddings, so we cannot rule out that a different algorithm would produce different—possibly tighter or looser—functional groupings and a correspondingly different Optimization Gap. Relatedly, the reported results depend on two fixed model choices: the sentence embedding model (`all-mpnet-base-v2`) used to vectorize LLM-generated descriptions, and the LLM used to generate those descriptions (Qwen2.5-30B-Instruct). We have not verified whether clustering outcomes are stable under alternative embedding models or alternative LLMs; different models could phrase functional descriptions with different granularity or emphasis, which could shift cluster boundaries and noise rates. We treat this as an open robustness question rather than an assumption, and outline concrete ablation plans in Section 8.

8 Future Work

We plan to use the enriched dataset resulting from this analysis as follows:

LLM-based automated refactoring (ICL). The golden reference Dockerfiles are high-quality, company-vetted exemplars in the exact technology stack of the target file. Providing them as few-shot in-context examples to an LLM-based repair system [11] eliminates the need for public-corpus retrieval and replaces it with semantically verified internal examples—directly addressing the context gap identified in [20]. Such a repair system could be integrated into CI pipelines or applied to all files with an optimization gap above a certain threshold as a batch remediation effort via pull requests.

Missing Golden Reference Resolution. Some clusters lack a viable internal reference ($S_{\max} < 0.75$). For these, we create a secure baseline configuration through manual engineering effort, guided by the cluster's functional domain and common base images. This new baseline then serves as the golden reference for all cluster members, enabling automated remediation for the 17% of clusters currently without a viable internal exemplar.

Longitudinal Debt Evolution and Operational Integration. The results in this paper are a static snapshot of a live, continuously evolving ecosystem: new Dockerfiles are added, existing ones are edited, and the company's hardened base images are periodically updated. Future work will operationalize the pipeline as a recurring process rather than a one-time study, along three complementary directions: (1) *periodic full re-clustering*, re-running Stages 4–6 on a fixed cadence (e.g., quarterly) to capture ecosystem-wide drift in workload composition and golden-reference quality; (2) *incremental assignment*, embedding and scoring newly added or modified Dockerfiles against existing cluster centroids between full re-clustering runs, flagging files that fit no known cluster as candidates for manual triage rather than forcing a match; and (3) *drift alerting*, monitoring when a cluster's current golden reference is superseded (e.g., because the company's hardened base image changes) or when a cluster's mean Optimization Gap increases, triggering re-evaluation of that cluster's reference. Together these mechanisms would reveal the velocity of debt accumulation, the impact of remediation efforts, and the lifecycle dynamics of infrastructure as code in an enterprise context, while keeping the golden-reference set current as the ecosystem evolves.

Deployment and Adoption Plan. Translating the 60.4% optimization gap into realized security improvement requires an adoption process, not merely the identification of golden references. We plan to prioritize outreach using the same ranking that drives Table 5: clusters with the largest cumulative optimization gap and a viable internal reference are contacted first, since they offer the highest expected security return for the lowest engineering effort. For each prioritized cluster, we plan to share the golden reference and a per-file diff against it with the owning team through existing inner-source communication channels (e.g., merge requests against the team's own repository), rather than mandating adoption centrally. Clusters without a viable reference (Table 6) are handled separately, as targeted engineering investment rather than propagation. A follow-up measurement—re-running the Stage 3 scoring on the same repositories after an adoption window—would let us report actual, rather than theoretical, improvement, complementing the longitudinal re-analysis described above.

Clustering and Model Robustness Ablations. To close the methodological gaps identified in Section 7, we plan a systematic robustness study along three axes: (1) *clustering algorithm*—re-clustering the same semantic embeddings with k-means, agglomerative, and spectral clustering, comparing cluster count, noise rate, base-image entropy, and package overlap against the HDBSCAN results reported in Table 3; (2) *embedding model*—repeating the pipeline with alternative sentence-embedding models to test whether the 251-cluster structure and the 60.4% optimization gap are an artifact of `all-mpnet-base-v2` or generalize across embedding choices; and (3) *description-generation LLM*—regenerating functional descriptions with additional open-source models beyond Qwen2.5-30B-Instruct and measuring the resulting cluster agreement (e.g., via Adjusted Rand Index) to quantify how sensitive the semantic grouping is to the specific LLM used. This ablation will let us report, rather than assume, the stability of our clustering methodology across algorithmic and model choices.

9 Conclusion

We presented an automated pipeline that quantifies Dockerfile security technical debt in enterprise inner-source repositories and identifies high-quality configurations as remediation targets. Applied to 11,470 Dockerfiles from a single large industrial company, the pipeline reveals a **60.4% relative improvement** in security posture achievable without developing new templates—simply by propagating internal best practices that already exist within each functional cluster. A conservative P90 target still yields a 28.75% global improvement. As with any single-organization study, the exact magnitude of these figures reflects this company's specific technology mix, governance posture, and technical-debt distribution; other enterprises should expect the qualitative pattern—latent internal references outnumbering the need for newly authored templates—to recur, but the precise percentages are not a priori transferable without a comparable study conducted in their own inner-source ecosystem.

The three research questions are answered with actionable precision. **RQ1**: 99% misconfiguration rate, 80.8% smell prevalence, median recency of 838 days, and a mean score of 0.48 establish a systemic, pervasive deficit that is empirically documented for the first time in a corporate inner-source context. **RQ2**: LLM-semantic clustering reduces global base-image entropy by 62% with only 17% noise, outperforming content-based and HLS syntactic baselines in functional cohesion; the force-directed graph confirms workload isolation across six distinct technology stacks. **RQ3**: 60.4% of measured debt is addressable from existing internal data without commissioning new secure templates.

These findings provide infrastructure teams with a data-driven prioritization framework, establish the enriched dataset as a retrieval corpus for LLM-based automated repair, and quantify—for the first time in an enterprise context—the scale of container security debt that automated software engineering can resolve.

Data Availability Statement

The dataset analyzed in this study consists of Dockerfiles and associated metadata collected from a proprietary enterprise inner-source environment. Due to licensing restrictions and confidentiality agreements, these artifacts cannot be released publicly or shared with external entities. The inner-source license governing these repositories explicitly prohibits external distribution. As a result, the dataset that led to the results in this paper is not available for review or public access.

The analysis pipeline is likewise not publicly released in source-code form. The implementation contains proprietary integrations with an internal GitLab instance, company-specific credential management, and internal toolchain orchestration that are subject to corporate intellectual property restrictions. To support replication in other enterprise contexts, the complete methodology—including all hyperparameters, scoring formulas, prompt template, normalization thresholds, and algorithmic choices—is fully specified in Section 4, enabling independent reimplementation without access to the original codebase.

Acknowledgments

The authors used GitHub Copilot and Claude (Anthropic) to assist with language improvement and document structuring during the preparation of this manuscript. All scientific content, results, and conclusions are solely the work of the authors.

References

[1] Erin Bank, Georg Grütter, Robert Hanmer, Klaas-Jan Stol, Padma Sudarsan, Cedric Williams, Tim Yao, and Nick Yeates. 2017. Innersource patterns for collaboration. In *Proceedings of the 24th Conference on Pattern Languages of Programs* (Vancouver, British Columbia, Canada) *(PLoP '17)*. The Hillside Group, USA, Article 24, 15 pages. doi:10.5555/3290281.3290310

[2] Stefan Buchner and Dirk Riehle. 2023. The Business Impact of Inner Source and How to Quantify It. *ACM Comput. Surv.* 56, 2, Article 47 (Sept. 2023), 27 pages. doi:10.1145/3611648

[3] Quang-Cuong Bui, Malte Laukötter, and Riccardo Scandariato. 2023. DockerCleaner: Automatic Repair of Security Smells in Dockerfiles. In *Proceedings of the IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, Bogotá, Colombia, 160–170. doi:10.1109/ICSME58846.2023.00026

[4] Ricardo J. G. B. Campello, Davoud Moulavi, and Jörg Sander. 2013. Density-based Clustering Based on Hierarchical Density Estimates. In *Advances in Knowledge Discovery and Data Mining (PAKDD)*. Springer, Gold Coast, Australia, 160–172. doi:10.1007/978-3-642-37456-2_14

[5] Jürgen Cito, Gerald Schermann, John Erik Wittern, Philipp Leitner, Sali Zumberi, and Harald C. Gall. 2017. An Empirical Analysis of the Docker Container Ecosystem on GitHub. In *Proceedings of the 14th International Conference on Mining Software Repositories (MSR)*. IEEE, Buenos Aires, Argentina, 323–333. doi:10.1109/MSR.2017.67

[6] Cloud Native Computing Foundation. 2024. *CNCF Annual Survey 2024.* Industry Report. Cloud Native Computing Foundation. https://www.cncf.io/reports/cncf-annual-survey-2024/

[7] Thomas Durieux. 2024. Empirical Study of the Docker Smells Impact on the Image Size. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*. ACM, Lisbon, Portugal, 1–12. doi:10.1145/3597503.3639143

[8] Kalvin Eng and Abram Hindle. 2021. Revisiting Dockerfiles in Open Source Software Over Time. In *Proceedings of the 18th International Conference on Mining Software Repositories (MSR)*. IEEE, Madrid, Spain, 449–459. doi:10.1109/MSR52588.2021.00057

[9] Mubin Ul Haque and M. Ali Babar. 2022. Well Begun is Half Done: An Empirical Study of Exploitability & Impact of Base-Image Vulnerabilities. In *Proceedings of the IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, Honolulu, HI, USA, 1066–1077. doi:10.1109/SANER53432.2022.00124

[10] Emna Ksontini, Marouane Kessentini, Thiago do N. Ferreira, and Foyzul Hassan. 2021. Refactorings and Technical Debt in Docker Projects: An Empirical Study. In *Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, Melbourne, Australia, 781–791. doi:10.1109/ASE51524.2021.9678585

[11] Emna Ksontini, Meriem Mastouri, Rania Khalsi, and Wael Kessentini. 2025. Refactoring for Dockerfile Quality: A Dive into Developer Practices and Automation Potential. In *Proceedings of the 22nd International Conference on Mining Software*

Repositories (MSR). IEEE, Ottawa, ON, Canada, 788–800. doi:10.1109/MSR66628.2025.00116

[12] Changyuan Lin, Sarah Nadi, and Hamzeh Khazaei. 2020. A Large-scale Data Set and an Empirical Study of Docker Images Hosted on Docker Hub. In *Proceedings of the IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, Adelaide, Australia, 371–381. doi:10.1109/ICSME46990.2020.00043

[13] Ruben Opdebeeck, Jonas Lesy, Ahmed Zerouali, and Coen De Roover. 2023. The Docker Hub Image Inheritance Network: Construction and Empirical Insights. In *Proceedings of the 23rd International Working Conference on Source Code Analysis and Manipulation (SCAM)*. IEEE, Bogotá, Colombia, 198–208. doi:10.1109/SCAM59687.2023.00029

[14] Giovanni Rosa, Emanuela Guglielmi, Mattia Iannone, Simone Scalabrino, and Rocco Oliveto. 2025. Mining and measuring the impact of change patterns for improving the size and build time of Docker images. *Empirical Software Engineering* 30, 5 (2025), 150. doi:10.1007/s10664-025-10680-8

[15] Giovanni Rosa, Antonio Mastropaolo, Simone Scalabrino, Gabriele Bavota, and Rocco Oliveto. 2023. Automatically Generating Dockerfiles via Deep Learning: Challenges and Promises. In *Proceedings of the IEEE/ACM International Conference on Software and System Processes (ICSSP)*. IEEE, Melbourne, Australia, 1–12. doi:10.1109/ICSSP59042.2023.00011

[16] Giovanni Rosa, Federico Zappone, Simone Scalabrino, and Rocco Oliveto. 2024. Fixing Dockerfile smells: an empirical study. *Empirical Software Engineering* 29, 5 (2024), 108. doi:10.1007/s10664-024-10471-7

[17] Taha Shabani, Noor Nashid, Parsa Alian, and Ali Mesbah. 2025. Dockerfile Flakiness: Characterization and Repair. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering*. IEEE Press, Ottawa, ON, Canada, 1793–1805. doi:10.1109/ICSE55347.2025.00238

[18] Hequan Shi, Lingyun Ying, Libo Chen, Haixin Duan, Ming Liu, and Zhi Xue. 2025. Dr. Docker: A Large-Scale Security Measurement of Docker Image Ecosystem. In *Proceedings of the ACM on Web Conference (WWW)*. ACM, Sydney, Australia, 2813–2823. doi:10.1145/3696410.3714653

[19] Yiwen Wu, Yang Zhang, Tao Wang, and Huaimin Wang. 2020. Characterizing the Occurrence of Dockerfile Smells in Open-Source Software: An Empirical Study. *IEEE Access* 8 (2020), 34127–34139. doi:10.1109/ACCESS.2020.2973750

[20] Ziyang Ye, Triet Huynh Minh Le, and M. Ali Babar. 2025. LLMSecConfig: An LLM-Based Approach for Fixing Software Container Misconfigurations. In *Proceedings of the 22nd International Conference on Mining Software Repositories (MSR)*. IEEE, Ottawa, ON, Canada, 629–641. doi:10.1109/MSR66628.2025.00099

[21] Yinyuan Zhang, Yang Zhang, Xinjun Mao, Yiwen Wu, Bo Lin, and Shangwen Wang. 2022. Recommending Base Image for Docker Containers based on Deep Configuration Comprehension. In *Proceedings of the IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, Honolulu, HI, USA, 449–453. doi:10.1109/SANER53432.2022.00060